

Programação em Python

Unidade 1

Desenvolver Algoritmos Utilizando Lógica de Programação

Sessão 03

Algoritmos

Lembrando da construção de um algoritmo...

Passos a seguir:

1. Definir o problema.
2. Executar uma análise preliminar.
3. Criar uma solução.
4. Aplicar o teste de qualidade.
5. Refinar a solução (Alteração).
6. Apresentar o produto final.

Algoritmos

Lembrando das fases de execução...

1. Coleta de informações de que o algoritmo precisa.
2. Processamento.
3. Apresentação do resultado.

Algoritmos

Tipos de processamento

- Tipos de Processamento:
 - *Processamento Sequencial*
 - Passos executados uma após o outro sem desvios.
 - *Processamento Condicional*
 - Conjunto de instruções executado ou não.
 - A execução depende de uma condição.
 - A condição pode definir uma resposta verdadeira ou falsa.
 - *Processamento com Repetição*
 - Conjunto de instruções que é executado um determinado número de vezes.

Algoritmos

Processamento condicional

- A execução depende de uma condição.
- A condição pode definir uma resposta verdadeira ou falsa.
- Conhecido, também, como “Fluxo de Decisão”.
- Execução definida a partir de testes utilizando operadores relacionais e/ou lógicos.

Algoritmos

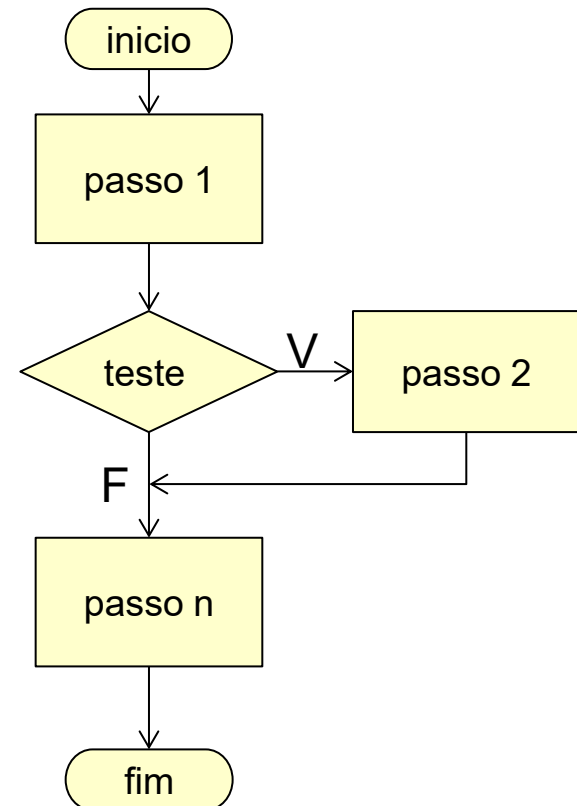
Fluxo de decisão simples

se <condição>

entao

<bloco de código>

fim-se



Algoritmos

Fluxo de decisão simples

Ex.:

Ler o ano de fabricação e o valor de um produto, calcular e apresentar o valor de desconto possível.

Caso o valor do produto seja maior que 1000, o desconto será equivalente à 20.

inicio

desconto <- 0

ler anoFabricacaoProduto

ler valorProduto

se valorProduto > 1000

entao

desconto <- valorProduto * 0.2

fim-se

escrever "O desconto é igual a ",
desconto

fim

Algoritmos

Fluxo de decisão composto

se <condição>

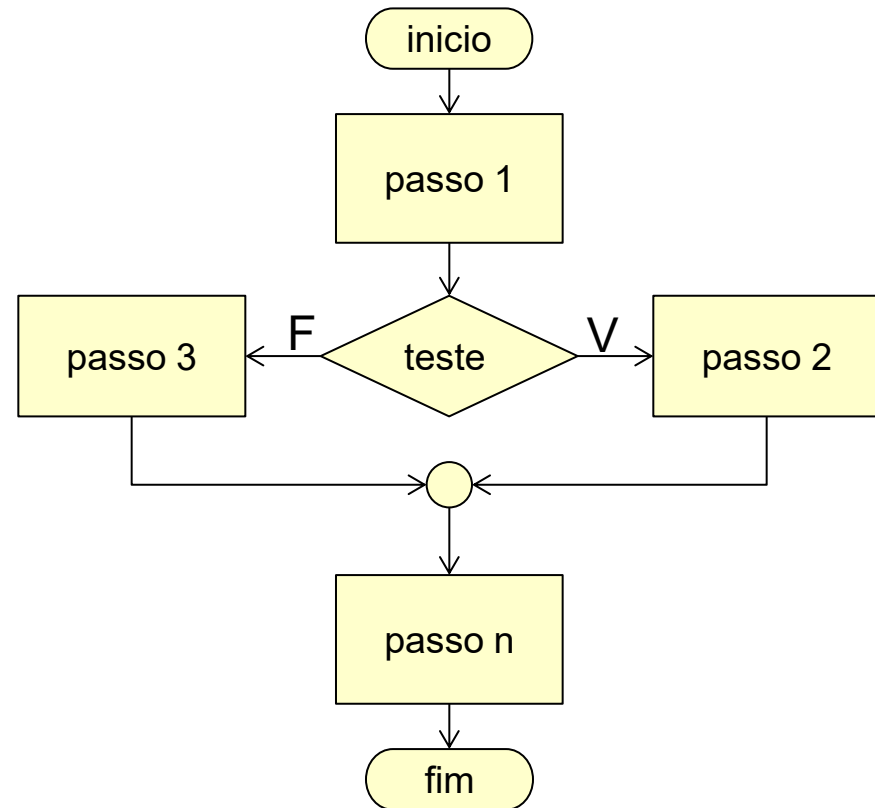
entao

<bloco de código>

senao

<bloco de código>

fim-se



Algoritmos

Fluxo de decisão composto

Ex.:

Ler o ano de fabricação e o valor de um produto, calcular e apresentar o valor de desconto possível.

O valor do desconto será igual a 20% do valor do produto quando este valor for maior que 1000, caso contrário, o desconto será equivalente à 10% sobre o valor do produto.

inicio

desconto <- 0

ler anoFabricacaoProduto

ler valorProduto

se valorProduto > 1000

entao

desconto <- valorProduto * 0.2

senao

desconto <- valorProduto * 0.1

fim-se

escrever "O desconto é igual a ",
desconto

fim

Algoritmos

Fluxo de decisão encadeado

se <condição>

entao

se <outra condição>

entao

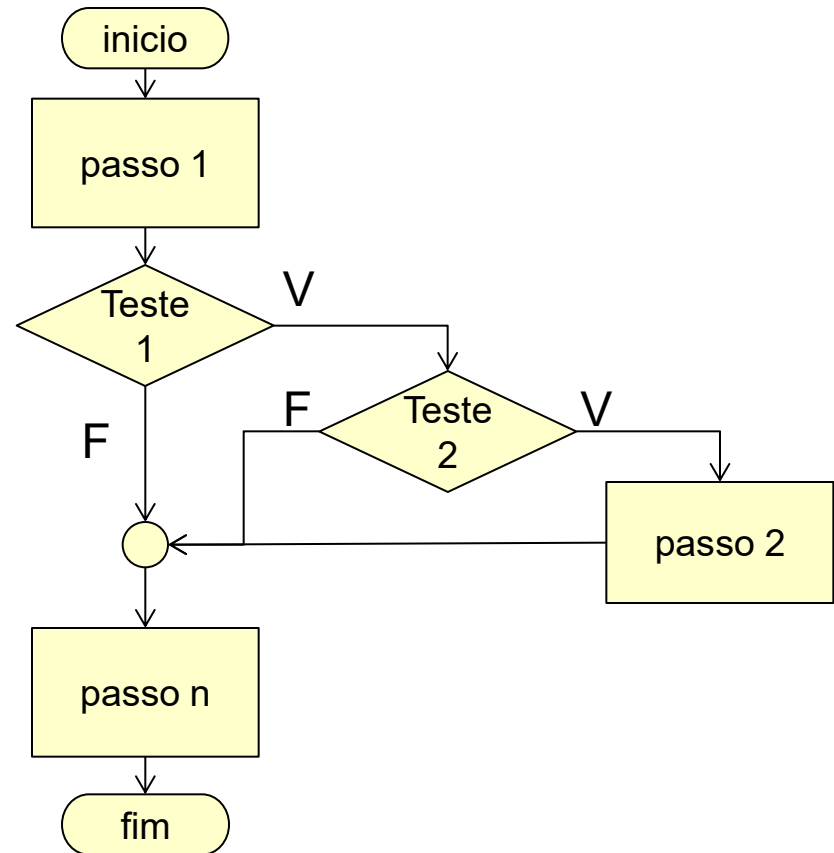
<bloco de código>

fim-se

senao

<bloco de código>

fim-se



Algoritmos

Fluxo de decisão composto

Ex.:

Ler o ano de fabricação e o valor de um produto, calcular e apresentar o valor de desconto possível.

Se o valor do produto for maior ou igual a 1000, um desconto de 25% deverá ser aplicado quando seu ano de fabricação for maior que 2000, caso contrário, o desconto deverá ser igual a 20%.

Caso o valor do produto não seja maior ou igual a 1000, deverá ser aplicado um desconto de 10% sobre esse valor.

inicio

ler anoFabricacaoProduto

ler valorProduto

se valorProduto >= 1000

entao

se anoFabricacaoProduto > 2000

entao

desconto <- valorProduto * 0.25

senao

desconto <- valorProduto * 0.2

fim-se

senao

desconto <- valorProduto * 0.1

fim-se

escrever "O desconto é igual a ", desconto

fim

Algoritmos

Fluxo de decisão “Escolha-Caso”

escolha <expressão>

caso valor1 **faça**

<bloco de código>

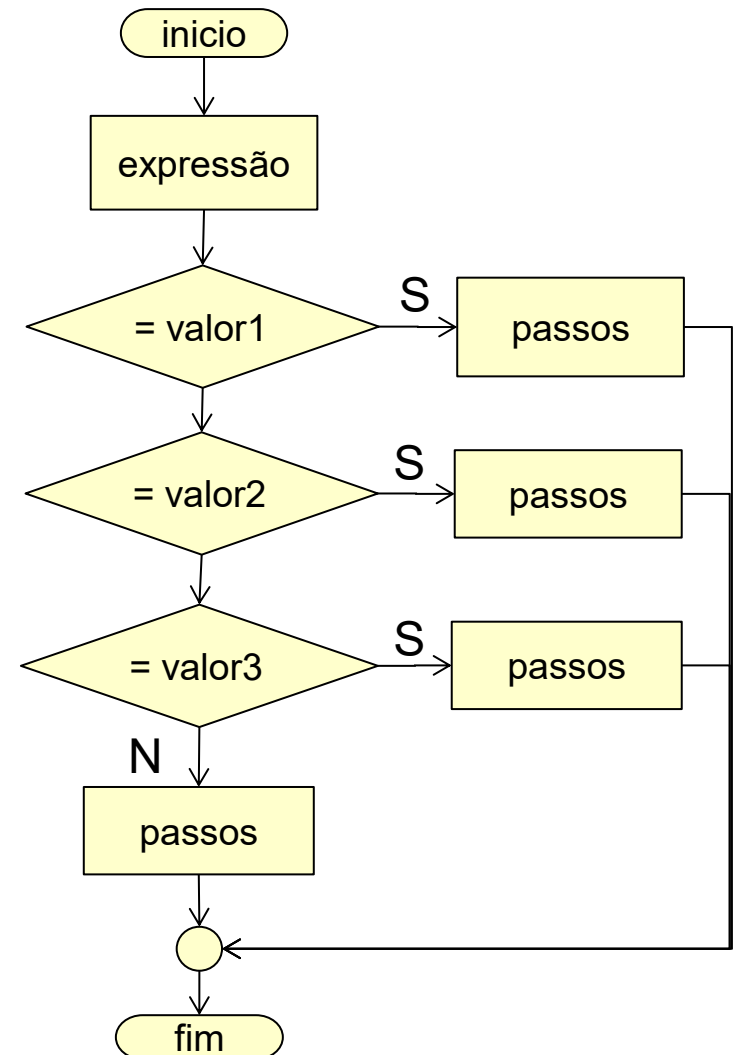
caso valor2 **faça**

<bloco de código>

outro caso (Obs.: Qualquer valor diferente de 1 e 2)

<bloco de código>

fim-escolha



Pyhton

Formatando a saída com *print()*

- Exemplo de saída

*O salário de Maria no mês
de abril foi R\$ 10500.50*

Código Python

```
nome = "Maria"
```

```
mes = "abril"
```

```
salario = 10500.50
```

```
print("O salário de {0} no mês de {1} foi
```

```
R$ { 3 : 2f }").format( nome, mes, salario ) )
```

float (real) :*nf*

n – casas decimais com arredondamento

f – tipo float

int (inteiro) :*nd*

n – espaços ou caractere de preenchimento

d – tipo inteiro

Forma Geral:

:[preenchimento][tipo alinhamento][largura].[precisão]

**qualquer
caracter**

**< esquerda
> direita
^ centro**

**largura
mínima
do campo**

**largura
máxima
do campo**

Python

Sobre tipos de dados

Tipos de dados	Descrição
Texto	str
Numérico	int, float, complex
Sequência	list, tuple, range
Mapa	dict
Set	set, frozenset
Lógico	bool
Binário (Binary)	bytes, bytearray, memoryview
Tipo "None"	NoneType

Python

Strings

- Array de bytes de caracteres Unicode
- Índice baseado em zero – Zero-based
- Comando *for* permite iteração
 - Ex.:

```
for x in "abacaxi"  
    print( x )
```
- Tamanho da string com a função *len()*
 - Ex.:

```
print( len( "abacaxi" ) )
```
- Instrução *in* ou *not in* para verificar a existência de uma string em outra
 - Ex.:

```
print( "ferro" in "ferrovia" )  
print( "casa" not in "ferrovia" )
```



Saiba mais

- Sobre métodos da String, visite:
https://www.w3schools.com/python/python_strings_methods.asp
- Sobre UNICODE, visite:
<https://www.ime.usp.br/~pf/algoritmos/apend/unicode.html>

Python

Sobre tipos e conversão de dados (casting)

- Tipo de dado da variável com a função *type()*
 - Ex.:

```
x = 10 // inteiro  
print( type ( x ) )
```
 - Resultado: `<class 'int'>`
- Casting de tipos numéricos e strings
 - String
 - Ex.:

```
print ( "Numero + str( 100 ) )
```
 - Integer – *int()*
 - Ex.:

```
print ( int( "2" ) )
```
 - Float – *float()*
 - Ex.:

```
print ( float( "2.10" ) )
```



Saiba mais

- Sobre tipos de dados, visite:
https://www.w3schools.com/python/python_datatypes.asp
- Sobre conversão de dados (casting), visite:
https://www.w3schools.com/python/python_numbers.asp

Pyhton

Processamento Condicional

- Simples

if condição:

<bloco de código>

<bloco de código pós condição>

- Composta

if condição:

<bloco de código>

else:

<bloco de código>

<bloco de código pós condição>

- Composta aninhada

if condição1:

<bloco de código>

elif condição2:

<bloco de código>

else:

<bloco de código>

<bloco de código pós condição>

Pyhton

Processamento Condicional - CASO

- Antes do Python 3.10

```
if condição1:  
    <bloco de código>  
elif condição2:  
    <bloco de código>  
else:  
    <bloco de código>  
<bloco de código pós condição>
```

- Após o Python 3.10

```
match termo:  
    case pattern1:  
        <bloco de código>  
    case pattern2:  
        <bloco de código>  
    case _ :  
        <bloco de código>  
<bloco de código pós termo>
```

Pyhton

Processamento Condicional - CASO

- Exemplo com `match`

```
linguagem = input( "Qual linguagem de programação você está aprendida?" )
```

```
match linguagem:
```

```
    case "JavaScript": print( "Você é um desenvolvedor Web" )
```

```
    case "Python": print( "Você é um cientista de dados" )
```

```
    case "PHP": print( "Você é um desenvolvedor back-end" )
```

```
    case "Java": print( "Você é um desenvolvedor massa" )
```

```
    case _: print( "Você é um desenvolvedor" )
```

```
<bloco de código pós condição>
```

Python

Tabela Verdade

Operando1	Operando2	E / and	OU / or	Não Operando1 / not Operando1
V	V	V	V	F
V	F	F	V	F
F	V	F	V	V
F	F	F	F	V

Programação em Python

Botafogo



Python

Praticando

Problemas:

1. Crie um programa para cada exercício da lista de exercícios de seleção.



Saiba mais

Referências sobre Python:

- <https://docs.python.org/3/tutorial/controlflow.html>
- <https://www.w3schools.com/python/>

Algoritmos/Python

- Considerações finais.
- Agradecimentos.



Programação em Python

Botafogo



Até a próxima!

